

Absolutely. **Batch Apex is the most important one to understand structurally**, because unlike Future and Queueable, a Batch class has **three required methods** and processes records in chunks.

We'll go from **zero** → **syntax** → **every keyword** → **simple question** → **realistic question** → **test class** → **line-by-line explanation**.

Batch Apex — Code From Zero

1. First understand what Batch Apex does

Suppose you have:

10,000 Accounts

and you want to update all of them.

Instead of:

```
10,000 records
      ↓
one huge operation
```

Batch Apex does:

```
10,000 records
      ↓
-----
Batch
-----
      ↓
200 records → execute()
200 records → execute()
200 records → execute()
...
      ↓
finish()
```

So the fundamental purpose of Batch Apex is:

Process a large number of records in smaller chunks.

2. The Batch Apex structure

Memorize this structure first:

apex

```
public class MyBatchClass implements Database.Batchable<SObject> {

    public Database.QueryLocator start(
        Database.BatchableContext bc
    ) {

        // Find records

    }

    public void execute(
        Database.BatchableContext bc,
        List<SObject> scope
    ) {

        // Process records

    }

    public void finish(
        Database.BatchableContext bc
    ) {

        // After everything finishes

    }

}
```

There are **three major methods**:

```
start()
execute()
finish()
```

You **MUST** understand these three.

3. `implements Database.Batchable<SObject>`

This is the first important keyword.

apex

```
public class MyBatchClass
```

This tells Salesforce:

```
"This class is a Batch Apex class."
```

`Database.Batchable` is an interface provided by Salesforce.

When you implement it, Salesforce expects your class to provide:

```
start()  
execute()  
finish()
```

4. Why `<SObject>` ?

You may see:

```
apex
```

```
Database.Batchable<SObject>
```

`SObject` is the generic Salesforce record type.

It essentially means:

```
"This Batch class works with Salesforce records."
```

You can also make it more specific.

For example:

```
apex
```

```
Database.Batchable<Account>
```

or:

```
apex
```

```
Database.Batchable<Contact>
```

For learning, you'll often see:

```
apex
```

5. The three methods

Think of them like this:

```
start()
```

Which records should I process?

```
execute()
```

What should I do with those records?

```
finish()
```

What should I do after all records are processed?

So:

```
START
  ↓
Find records
  ↓
EXECUTE
  ↓
Process chunk
  ↓
EXECUTE
  ↓
Process chunk
  ↓
EXECUTE
  ↓
...
  ↓
FINISH
```

6. START METHOD

Let's look at it:

```
apex
```

```
public Database.QueryLocator start(  
    Database.BatchableContext bc  
) {  
  
}
```

There are several pieces.

`public`

apex

```
public Database.QueryLocator start(...)
```

Makes the method accessible.

`Database.QueryLocator`

This is the return type.

apex

```
Database.QueryLocator
```

It tells Salesforce:

"I'm returning a query that identifies the records this Batch should process."

For example:

apex

```
return Database.getQueryLocator(  
    'SELECT Id, Name FROM Account'  
);
```

7. `Database.getQueryLocator()`

This is extremely important.

apex

```
Database.getQueryLocator(  
    'SELECT Id, Name FROM Account'  
)
```

It tells Salesforce:

"Here are the records I want the Batch to process."

For example:

```
apex  
  
return Database.getQueryLocator(  
    'SELECT Id, Name FROM Account'  
);
```

means:

Get all Accounts.

8. Database.BatchableContext

You'll see:

```
apex  
  
Database.BatchableContext bc
```

This gives Salesforce context information about the current Batch execution.

For beginner questions, you usually don't need to use `bc`.

Just understand:

`bc` represents the Batch execution context.

9. EXECUTE METHOD

Now:

```
apex  
  
public void execute(  
    Database.BatchableContext bc,
```

```
List<SObject> scope
) {

}
```

This is where you process the records.

What is `scope` ?

This is VERY important.

apex

```
List<SObject> scope
```

contains the **current chunk of records**.

Suppose you have:

1,000 Accounts

and batch size:

200

Then:

```
execute()
  ↓
scope = Accounts 1-200

execute()
  ↓
scope = Accounts 201-400

execute()
  ↓
scope = Accounts 401-600
```

and so on.

10. `finish()`

Finally:

apex

```
public void finish(  
    Database.BatchableContext bc  
) {  
  
}
```

This executes **once after the entire batch finishes**.

For example:

apex

```
System.debug('Batch completed');
```

You can use `finish()` for things like:

- sending a completion email
- logging completion
- starting another process
- performing post-processing

11. Your first Batch Apex question

Let's start with a simple coding question.

Question:

Create a Batch Apex class that retrieves all Accounts and prints each Account's Name in the debug log.

12. Solution

apex

```
public class AccountBatch implements Database.Batchable<SObject> {  
  
    public Database.QueryLocator start(  
        Database.BatchableContext bc  
    ) {  
  
        return Database.getQueryLocator(  
            'SELECT Id, Name FROM Account'  
        );  
    }  
}
```

```

    );
}

public void execute(
    Database.BatchableContext bc,
    List<SObject> scope
) {

    for (SObject record : scope) {

        Account acc = (Account) record;

        System.debug(acc.Name);
    }
}

public void finish(
    Database.BatchableContext bc
) {

    System.debug('Batch completed');
}
}

```

Let's break this down.

13. Class declaration

apex

```

public class AccountBatch
    implements Database.Batchable<SObject>

```

Means:

Create a public Batch Apex class called `AccountBatch`.

14. `start()`

apex

```

public Database.QueryLocator start(
    Database.BatchableContext bc
)

```

This is required by Batch Apex.

Inside:

apex

```
return Database.getQueryLocator(  
    'SELECT Id, Name FROM Account'  
);
```

We tell Salesforce:

"Get all Accounts."

15. Why `return`?

Because:

apex

```
Database.QueryLocator
```

is the return type.

So we need to return a QueryLocator.

apex

```
return Database.getQueryLocator(...);
```

16. `execute()`

apex

```
public void execute(  
    Database.BatchableContext bc,  
    List<SObject> scope  
)
```

This method receives one chunk of records.

For example:

scope

↓

```
Account 1
Account 2
Account 3
...
Account 200
```

17. Loop through scope

apex

```
for (SObject record : scope)
```

means:

Go through each record in this chunk.

18. Why cast to Account?

We declared:

apex

```
List<SObject> scope
```

So Salesforce sees each item as an `SObject`.

We know these are actually Accounts.

Therefore:

apex

```
Account acc = (Account) record;
```

converts it to an Account.

Then:

apex

```
System.debug(acc.Name);
```

prints the Account Name.

19. `finish()`

apex

```
public void finish(  
    Database.BatchableContext bc  
)
```

runs once after all chunks finish.

apex

```
System.debug('Batch completed');
```

20. How do we run a Batch?

This is VERY important.

We use:

apex

```
Database.executeBatch()
```

For example:

apex

```
Database.executeBatch(  
    new AccountBatch(),  
    200  
);
```

Let's understand this.

```
new AccountBatch()
```

Creates the Batch object.

```
Database.executeBatch()
```

Tells Salesforce:

| Execute this Batch.

| 200

This is the **batch size**.

Meaning:

Process up to 200 records per `execute()` invocation.

21. Visualize it

Suppose there are:

500 Accounts

and:

```
apex

Database.executeBatch(
    new AccountBatch(),
    200
);
```

Salesforce does approximately:

```
start()
  ↓
500 Accounts identified
  ↓
execute(scope = 200)
  ↓
execute(scope = 200)
  ↓
execute(scope = 100)
  ↓
finish()
```

That's Batch Apex.

22. Now let's make a REAL useful question

Question 2:

Create a Batch Apex class that updates all Accounts and changes their Description to `"Processed by Batch"`.

This is the type of question you should practice.

23. Batch class

apex

```
public class AccountUpdateBatch
    implements Database.Batchable<SObject> {

    public Database.QueryLocator start(
        Database.BatchableContext bc
    ) {

        return Database.getQueryLocator(
            'SELECT Id, Description FROM Account'
        );
    }

    public void execute(
        Database.BatchableContext bc,
        List<SObject> scope
    ) {

        List<Account> accounts = new List<Account>();

        for (SObject record : scope) {

            Account acc = (Account) record;

            acc.Description = 'Processed by Batch';

            accounts.add(acc);
        }

        update accounts;
    }

    public void finish(
        Database.BatchableContext bc
    ) {

        System.debug('Batch completed');
    }
}
```

24. Why do we create `accounts` ?

We write:

apex

```
List<Account> accounts = new List<Account>();
```

because we're going to collect all the Accounts from the current scope.

Then:

apex

```
accounts.add(acc);
```

adds each modified Account to the list.

Finally:

apex

```
update accounts;
```

updates them all in one DML statement.

This is **bulkified Apex**.

25. Could we update `scope` directly?

Yes, if the scope is typed appropriately.

For example, if your Batch is:

apex

```
Database.Batchable<Account>
```

then the execute method can be:

apex

```
public void execute(  
    Database.BatchableContext bc,  
    List<Account> scope  
)
```

Then the code becomes much cleaner.

Let's use this approach.

26. Better Account-specific Batch

apex

```
public class AccountUpdateBatch
    implements Database.Batchable<Account> {

    public Database.QueryLocator start(
        Database.BatchableContext bc
    ) {

        return Database.getQueryLocator(
            'SELECT Id, Description FROM Account'
        );
    }

    public void execute(
        Database.BatchableContext bc,
        List<Account> scope
    ) {

        for (Account acc : scope) {

            acc.Description = 'Processed by Batch';

        }

        update scope;
    }

    public void finish(
        Database.BatchableContext bc
    ) {

        System.debug('Batch completed');
    }
}
```

I recommend you learn this style first when the question is specifically about Accounts.

27. Why is this better?

Instead of:

apex

```
List<SObject> scope
```

we have:

apex

```
List<Account> scope
```

So Salesforce already knows:

These are Accounts.

We don't need:

apex

```
Account acc = (Account) record;
```

We can directly do:

apex

```
for (Account acc : scope)
```

28. Now the code is much easier

```
start()  
↓  
Query Accounts  
↓  
execute()  
↓  
Receive List<Account>  
↓  
Loop through Accounts  
↓  
Change Description  
↓  
update scope  
↓  
Next chunk
```

29. Running the Batch

Execute Anonymous:

```
apex

Database.executeBatch(
    new AccountUpdateBatch(),
    200
);
```

That's all.



30. Now the important part — Test Class

Here's the complete test:

```
apex

@Test
public class AccountUpdateBatchTest {

    @Test
    static void testAccountUpdateBatch() {

        List<Account> accounts = new List<Account>();

        for (Integer i = 0; i < 5; i++) {

            accounts.add(
                new Account(
                    Name = 'Test Account ' + i
                )
            );
        }

        insert accounts;

        Test.startTest();

        Database.executeBatch(
            new AccountUpdateBatch(),
            2
        );
    }
}
```

```

Test.stopTest();

List<Account> updatedAccounts = [
    SELECT Id, Description
    FROM Account
    WHERE Id IN :accounts
];

for (Account acc : updatedAccounts) {

    System.assertEquals(
        'Processed by Batch',
        acc.Description
    );
}
}
}

```

Now let's understand every part.

31. @isTest

apex

```
@isTest
```

Tells Salesforce:

This is test code.

32. Test class

apex

```
public class AccountUpdateBatchTest
```

Creates our test class.

Usually:

```

Production:
AccountUpdateBatch

```

Test:

33. Test method

apex

```
@isTest
static void testAccountUpdateBatch()
```

This is the method that tests our Batch.

34. Create test data

apex

```
List<Account> accounts = new List<Account>();
```

Creates an empty list.

Then:

apex

```
for (Integer i = 0; i < 5; i++)
```

runs five times.

We create:

```
Test Account 0
Test Account 1
Test Account 2
Test Account 3
Test Account 4
```

35. Add Accounts

apex

```
accounts.add(
    new Account(
        Name = 'Test Account ' + i
```

```
)  
);
```

Creates an Account and adds it to the list.

36. Insert

```
apex  
  
insert accounts;
```

Now the five Accounts actually exist in Salesforce.

37. Start Test

```
apex  
  
Test.startTest();
```

Now we start the asynchronous Batch operation.

38. Execute Batch

```
apex  
  
Database.executeBatch(  
    new AccountUpdateBatch(),  
    2  
);
```

This is extremely important.

We are saying:

Run `AccountUpdateBatch` with a batch size of 2.

Because we created 5 records:

```
5 Accounts
```

and batch size is:

the Batch can execute approximately:

```
execute() → 2 records  
execute() → 2 records  
execute() → 1 record
```

This is useful in a test because it helps us exercise multiple `execute()` invocations.

39. Stop Test

apex

```
Test.stopTest();
```

This causes the asynchronous Batch execution to complete within the test context.

This is why the sequence:

apex

```
Test.startTest();  
  
Database.executeBatch(...);  
  
Test.stopTest();
```

is so important.

40. Query updated records

apex

```
List<Account> updatedAccounts = [  
    SELECT Id, Description  
    FROM Account  
    WHERE Id IN :accounts  
];
```

We're retrieving the Accounts again.

We want to see whether the Batch changed them.

41. Loop through results

```
apex  
  
for (Account acc : updatedAccounts)
```

Check every Account.

42. Assert

```
apex  
  
System.assertEquals(  
    'Processed by Batch',  
    acc.Description  
);
```

We're saying:

Every Account's Description should now be "Processed by Batch".

If yes:

✓ Test passes.

If no:

✗ Test fails.

43. VERY IMPORTANT — scope

This is probably the most important Batch concept.

Suppose:

```
1000 Accounts
```

Batch size:

```
200
```

Your `execute()` doesn't receive all 1000.

It receives:

```
scope = 200
```

Then another:

```
scope = 200
```

etc.

So:

apex

```
public void execute(  
    Database.BatchableContext bc,  
    List<Account> scope  
)
```

means:

"Give me one chunk of Accounts at a time."

44. `start()` VS `execute()` VS `finish()`

Memorize this:

Method	Question it answers
<code>start()</code>	Which records?
<code>execute()</code>	What should I do with them?
<code>finish()</code>	What should happen after everything?

Example:

```
start()  
"Give me all Accounts with Status = Active"  
  
↓  
  
execute()  
"Change their Description"
```

```

↓

execute()
"Change next chunk"

↓

execute()
"Change next chunk"

↓

finish()
"Send completion email"

```

45. Example Question 3 — Conditional processing

Now let's make it more realistic.

Question:

Create a Batch Apex class that finds Accounts whose Rating is "Hot" and changes their Description to "Priority Account".

Solution

```

apex

public class HotAccountBatch
    implements Database.Batchable<Account> {

    public Database.QueryLocator start(
        Database.BatchableContext bc
    ) {

        return Database.getQueryLocator(
            'SELECT Id, Rating, Description ' +
            'FROM Account ' +
            'WHERE Rating = \'Hot\''
        );
    }

    public void execute(
        Database.BatchableContext bc,
        List<Account> scope
    ) {

```

```

        for (Account acc : scope) {

            acc.Description = 'Priority Account';

        }

        update scope;
    }

    public void finish(
        Database.BatchableContext bc
    ) {

        System.debug('Hot Account Batch completed');

    }
}

```

46. What's different?

The important part is:

apex

```
WHERE Rating = 'Hot'
```

We're only selecting the Accounts we care about.

So:

```

100,000 Accounts
    ↓
start()
    ↓
Only Hot Accounts
    ↓
execute()

```

47. Test for this

apex

```

@isTest
public class HotAccountBatchTest {

```

```

@Test
static void testHotAccountBatch() {

    Account hotAccount = new Account(
        Name = 'Hot Account',
        Rating = 'Hot'
    );

    Account coldAccount = new Account(
        Name = 'Cold Account',
        Rating = 'Cold'
    );

    insert new List<Account>{
        hotAccount,
        coldAccount
    };

    Test.startTest();

    Database.executeBatch(
        new HotAccountBatch(),
        1
    );

    Test.stopTest();

    List<Account> results = [
        SELECT Id, Rating, Description
        FROM Account
        WHERE Id IN :new List<Id>{
            hotAccount.Id,
            coldAccount.Id
        }
    ];

    for (Account acc : results) {

        if (acc.Rating == 'Hot') {

            System.assertEquals(
                'Priority Account',
                acc.Description
            );

        } else {

            System.assertEquals(
                null,
                acc.Description
            );

        }
    }
}

```

```
}  
}  
}
```

48. Why create both Hot and Cold?

Because we want to test the condition.

We want to prove:

```
Hot Account  
↓  
Updated
```

and:

```
Cold Account  
↓  
Not updated
```

This is much better testing than only testing the positive case.

49. `finish()` can do something useful

For example:

```
apex  
  
public void finish(Database.BatchableContext bc) {  
  
    System.debug('Batch completed');  
  
}
```

But in real applications, it could do:

```
Batch finishes  
↓  
Send completion email
```

or:

```
Batch finishes  
↓
```

50. How to remember Batch syntax

When you see:

"Create Batch Apex..."

Immediately write:

apex

```
public class MyBatch implements Database.Batchable<Account> {

    public Database.QueryLocator start(
        Database.BatchableContext bc
    ) {

        return Database.getQueryLocator(
            'SELECT Id FROM Account'
        );
    }

    public void execute(
        Database.BatchableContext bc,
        List<Account> scope
    ) {

        // logic

        update scope;
    }

    public void finish(
        Database.BatchableContext bc
    ) {

        // after batch

    }
}
```

Then execute:

apex

```
Database.executeBatch(
    new MyBatch(),
```



51. All Batch keywords you need to know

Keyword / Method	What it means
<code>public</code>	Accessible class/method
<code>class</code>	Creates a class
<code>implements</code>	Implements an interface
<code>Database.Batchable<Account></code>	Makes class a Batch for Accounts
<code>start()</code>	Determines records to process
<code>Database.BatchableContext</code>	Batch execution context
<code>Database.QueryLocator</code>	Represents records returned for Batch processing
<code>Database.getQueryLocator()</code>	Gets records from SOQL for the Batch
<code>execute()</code>	Processes each chunk
<code>scope</code>	Current chunk of records
<code>finish()</code>	Runs after all chunks complete
<code>Database.executeBatch()</code>	Starts the Batch
<code>200</code>	Example batch size
<code>Test.startTest()</code>	Starts test execution context
<code>Test.stopTest()</code>	Completes async execution during test
<code>System.assertEquals()</code>	Checks expected vs actual value



52. Batch vs Queueable — code difference

This is where you should connect today's lesson with the previous one.

Queueable

apex

```
public class MyQueueable implements Queueable {

    public void execute(QueueableContext context) {

        // logic

    }

}
```

Run:

apex

```
System.enqueueJob(
    new MyQueueable()
);
```

Batch

apex

```
public class MyBatch implements Database.Batchable<Account> {

    public Database.QueryLocator start(
        Database.BatchableContext bc
    ) {

        return Database.getQueryLocator(
            'SELECT Id FROM Account'
        );
    }

    public void execute(
        Database.BatchableContext bc,
        List<Account> scope
    ) {

        // logic

    }

    public void finish(
        Database.BatchableContext bc
    ) {
```

```
}  
}
```

Run:

```
apex  
  
Database.executeBatch(  
    new MyBatch(),  
    200  
);
```

The mental difference:

Queueable:

"Here's one asynchronous job."

Batch:

"Here's a large set of records. Process them chunk by chunk."

★ 53. One very important coding rule

Don't write:

```
apex  
  
for (Account acc : scope) {  
  
    update acc;  
  
}
```

Instead:

```
apex  
  
for (Account acc : scope) {  
  
    acc.Description = 'Processed';  
  
}  
  
update scope;
```

Why?

Because you're doing:

```
1 execute()  
  ↓  
200 records  
  ↓  
1 DML statement
```

instead of:

```
1 execute()  
  ↓  
200 records  
  ↓  
200 DML statements ❌
```

This is **bulkification**, and it's extremely important in Apex.

54. How to recognize a Batch question

If the question says:

"Process all Accounts..."

Think **Batch** if the data volume could be large.

If it says:

"Process thousands/millions of records..."

Definitely think **Batch**.

If it says:

"Process records in chunks..."

Definitely **Batch**.

If it says:

"Create a Batch Apex class..."

Immediately remember:

```
Database.Batchable
```

↓

```
start()
```

↓

```
execute()
```

↓

```
finish()
```

The Batch formula

This is the one I'd memorize before moving to Schedulable:

```
BATCH APEX
```

```
implements Database.Batchable<SObject>
```

↓

```
start()
```

```
"WHICH RECORDS?"
```

↓

```
execute()
```

```
"WHAT DO I DO?"
```

↓

```
execute()
```

```
"NEXT CHUNK"
```

↓

```
execute()
```

```
"NEXT CHUNK"
```

↓

```
finish()
```

```
"EVERYTHING DONE"
```

↓

```
Database.executeBatch()
```

And the **test** formula:

```
CREATE TEST DATA
↓
INSERT DATA
↓
Test.startTest()
↓
Database.executeBatch()
↓
Test.stopTest()
↓
QUERY UPDATED DATA
↓
System.assertEquals()
```

Once this is comfortable, **Schedulable Apex** is actually much shorter to code: it mainly introduces `implements Schedulable`, the `execute(SchedulableContext)` method, `System.schedule()`, and Cron expressions. The interesting part is then learning how to combine **Schedulable + Batch**, which is a very common real-world pattern.